Parte 1: Configuración Básica Pestaña Basics

Se accedió al portal de Azure y se inició la creación de una nueva Function App.

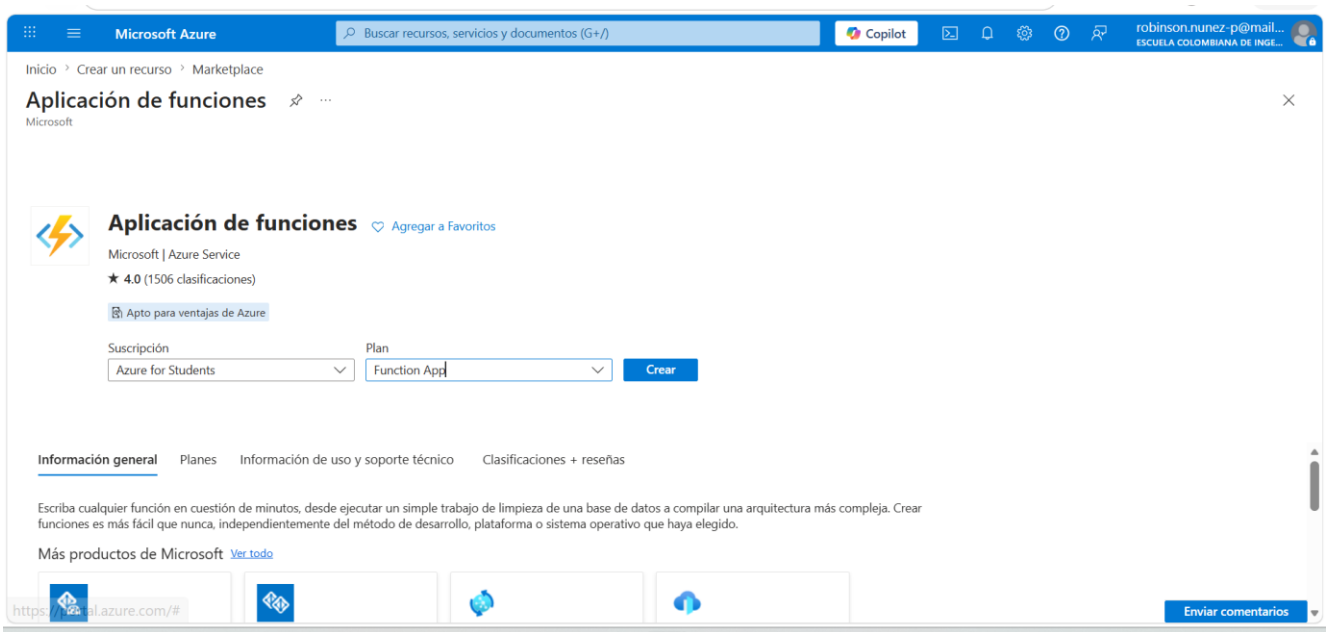
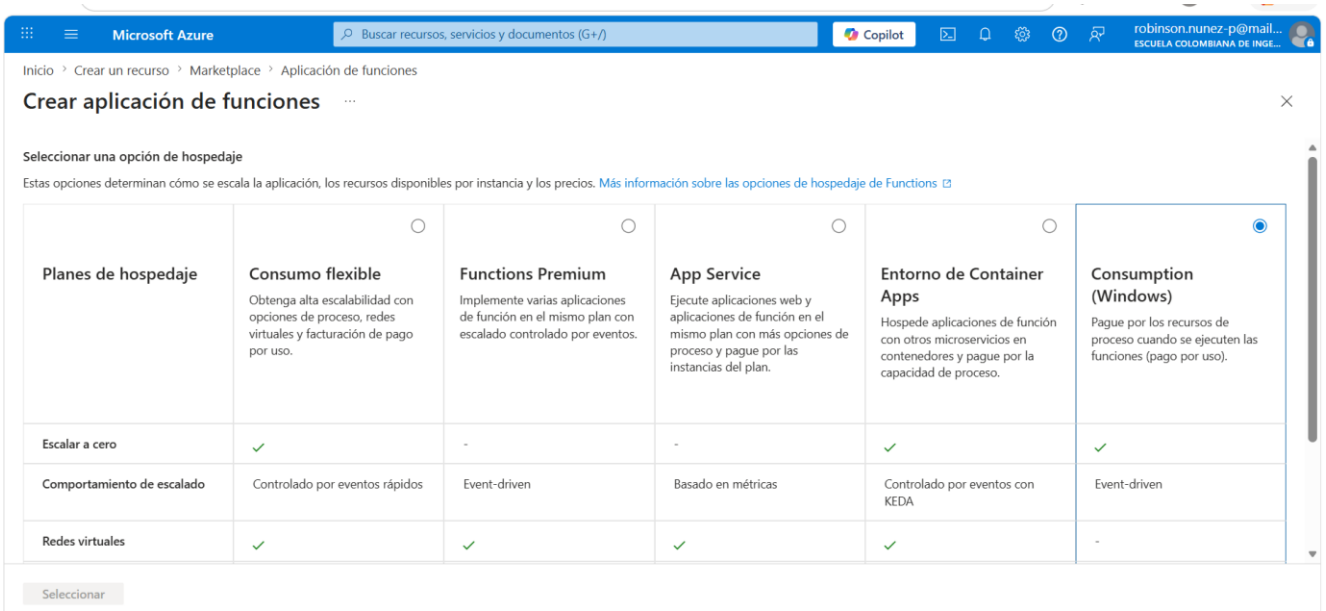
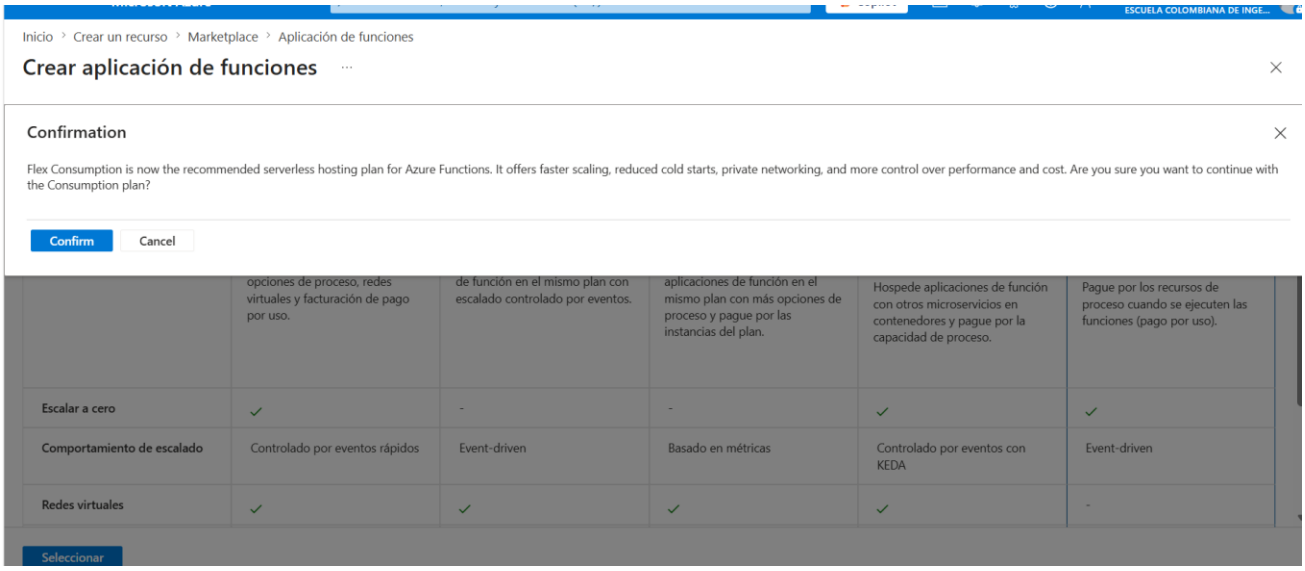


Tabla comparativa de los planes de hospedaje disponibles para Azure Functions elegimos el de Consumo Windows.

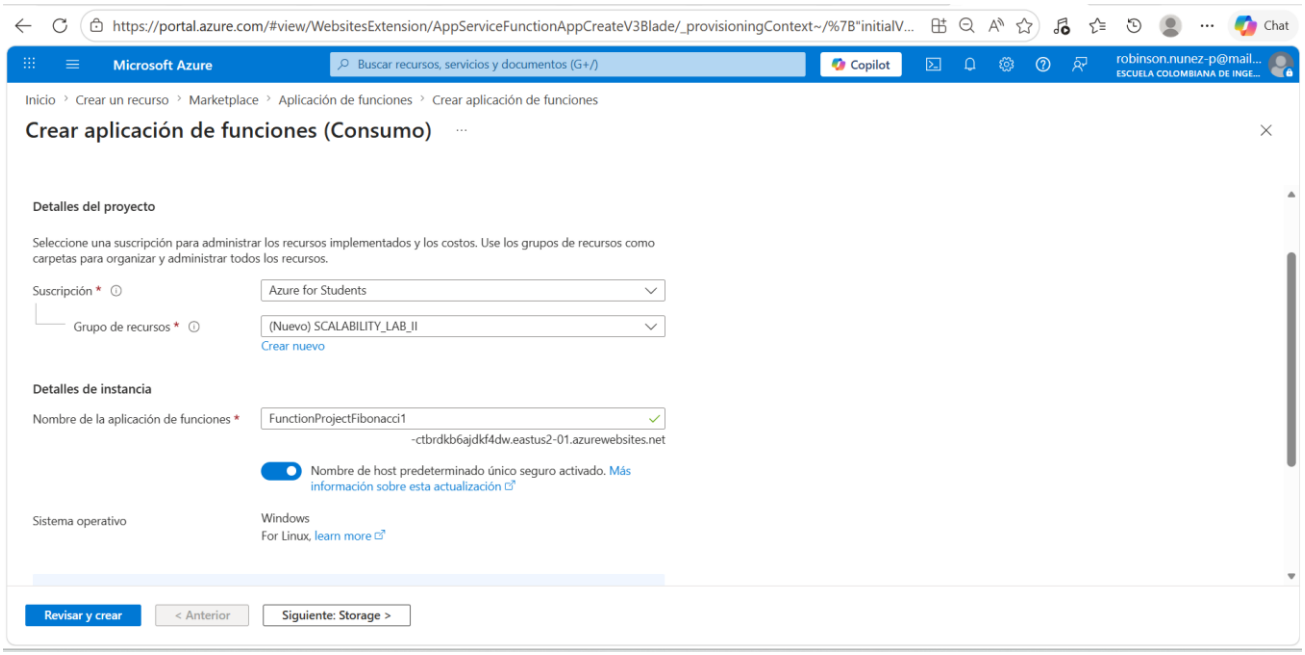


Planes de hospedaje	Consumo flexible	Functions Premium	App Service	Entorno de Container Apps	Consumption (Windows)
	Obtenga alta escalabilidad con opciones de proceso, redes virtuales y facturación de pago por uso.	Implemente varias aplicaciones de función en el mismo plan con escalado controlado por eventos.	Ejecute aplicaciones web y aplicaciones de función en el mismo plan con más opciones de proceso y pague por las instancias del plan.	Hospede aplicaciones de función con otros microservicios en contenedores y pague por la capacidad de proceso.	Pague por los recursos de proceso cuando se ejecuten las funciones (pago por uso).
Escalar a cero	✓	-	-	✓	✓
Comportamiento de escalado	Controlado por eventos rápidos	Event-driven	Basado en métricas	Controlado por eventos con KEDA	Event-driven
Redes virtuales	✓	✓	✓	✓	-

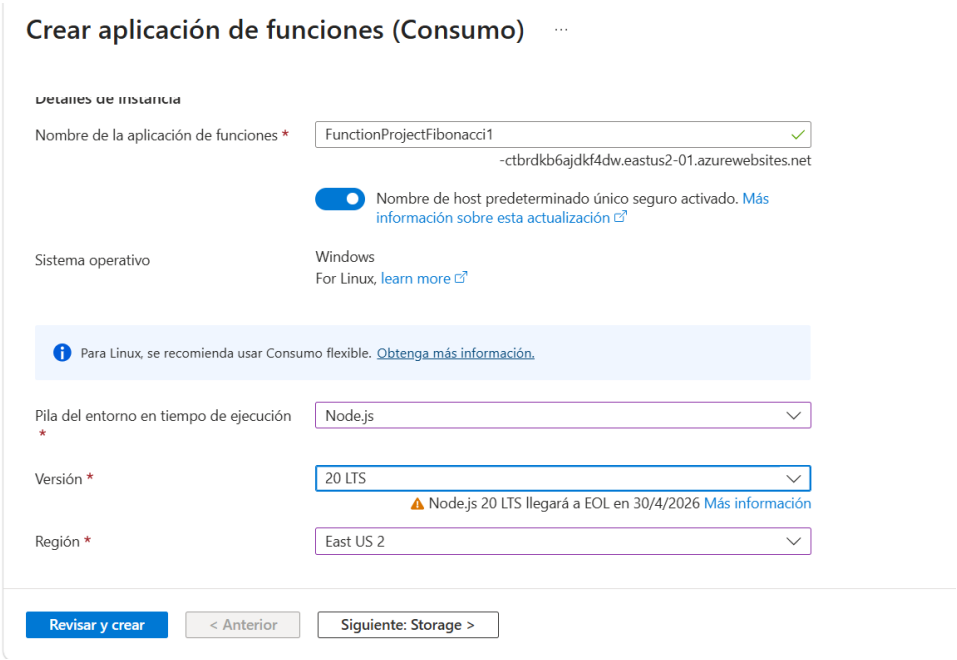
Continuamos con el plan de Consumo clásico.



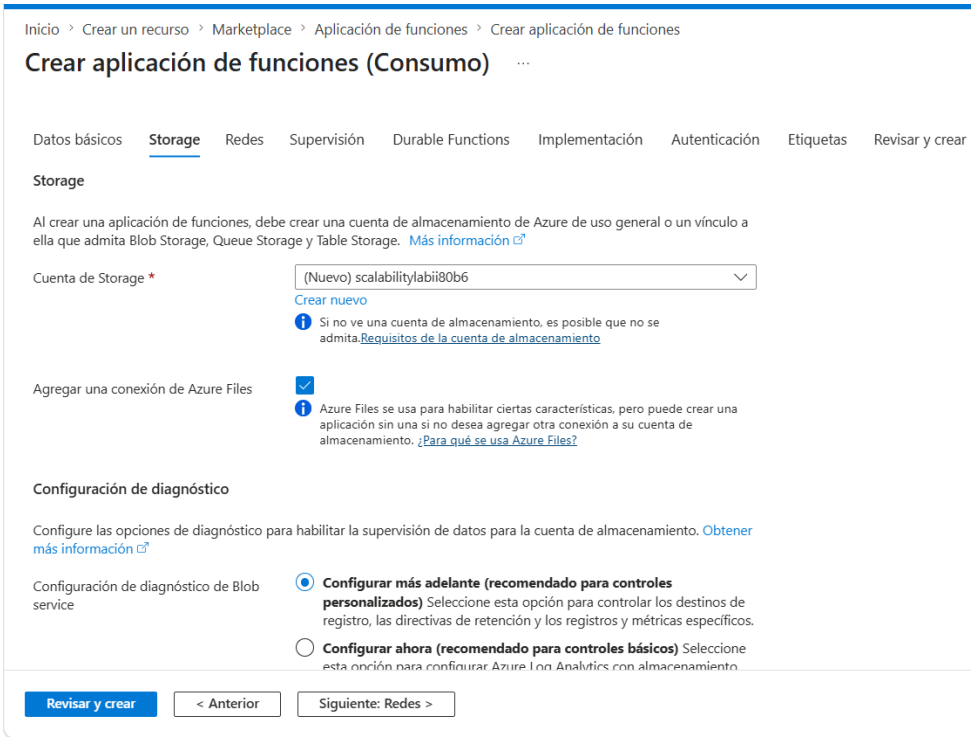
Llenamos el Formulario con los datos del proyecto: Suscripción Azure for Students, grupo de recursos 'SCALABILITY_LAB_II' y nombre 'FunctionProjectFibonacci1'



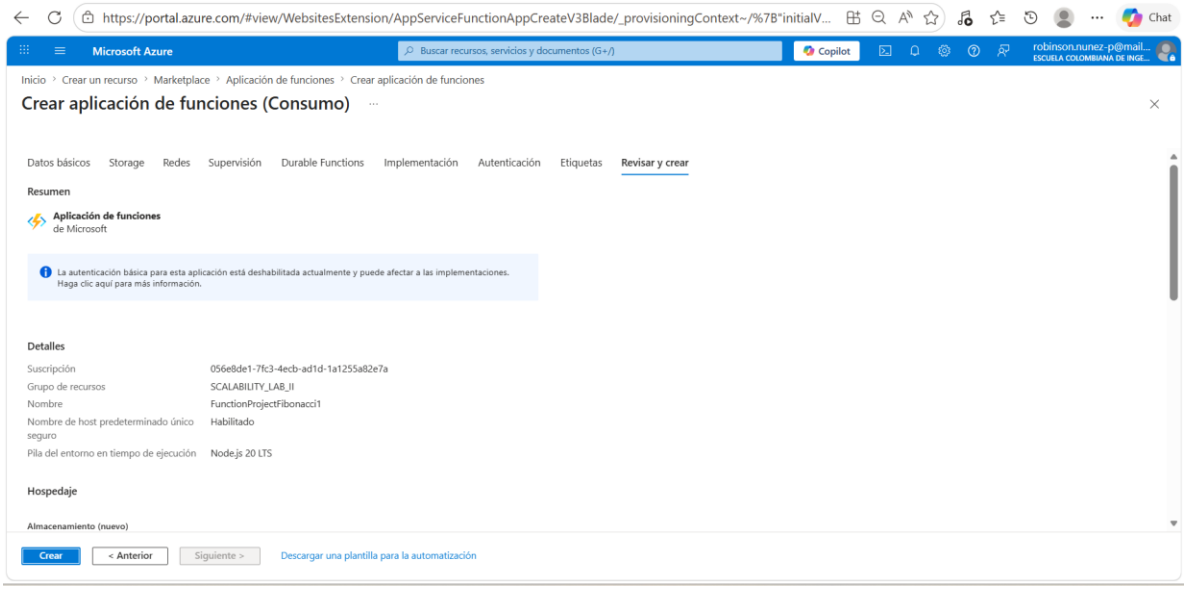
Continuación del formulario de creación donde se configuró el runtime Node.js versión 20 LTS y la región East US 2 para la Function App.



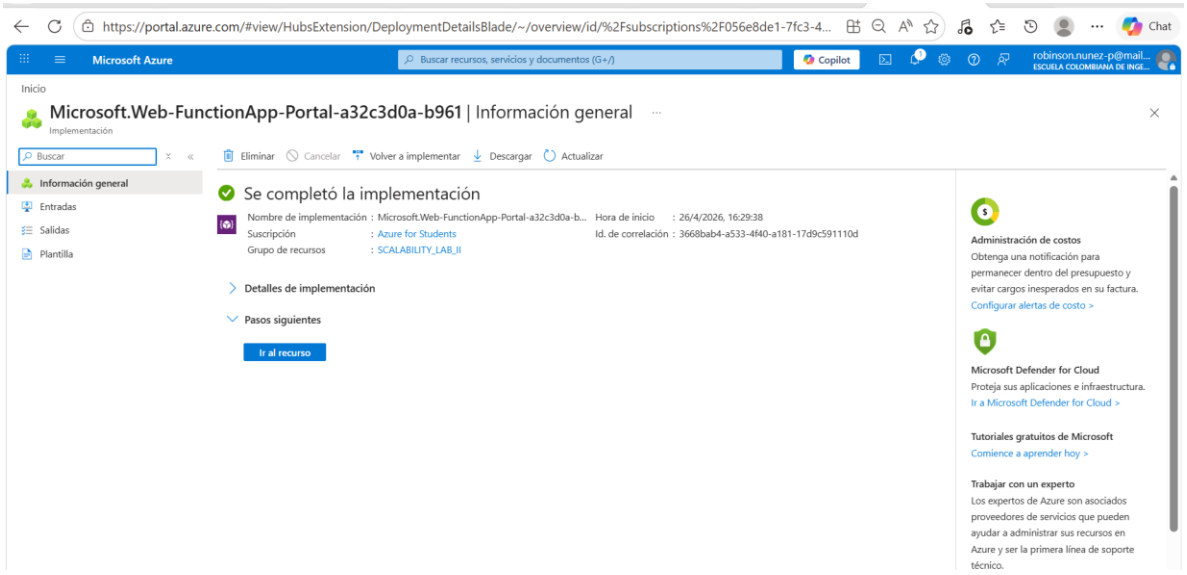
Se creó automáticamente la cuenta de almacenamiento scalabilitylabii80b6 requerida por el runtime de Azure Functions.



Validamos lo que sale en la pestaña de Revisar y crear mostrando el resumen completo

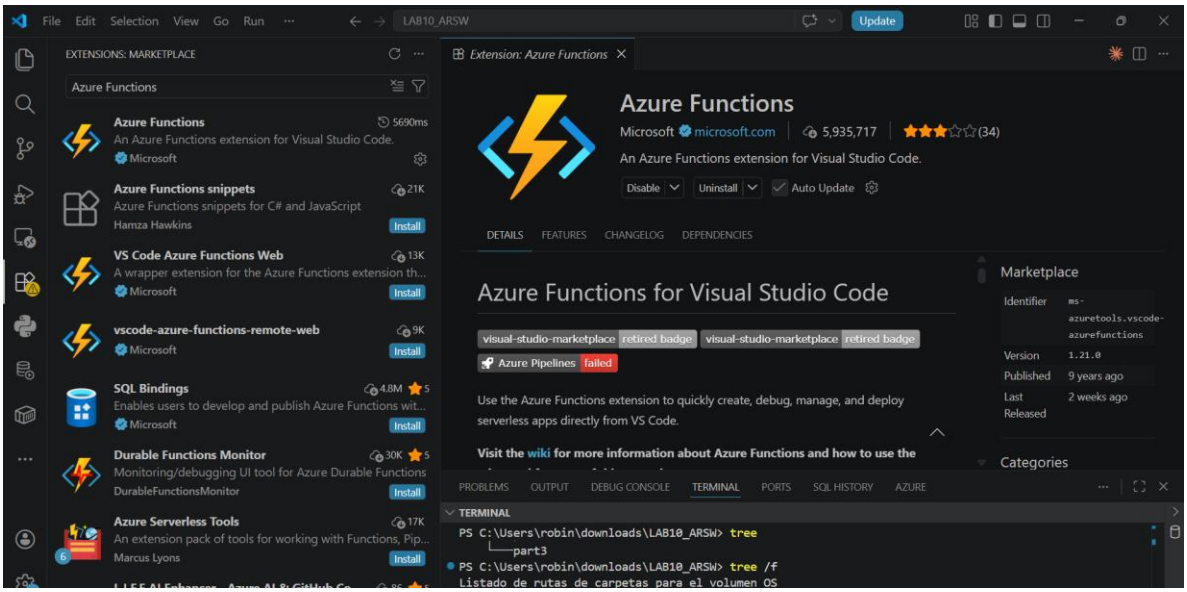


Se confirma el Portal implementado de la Function App.

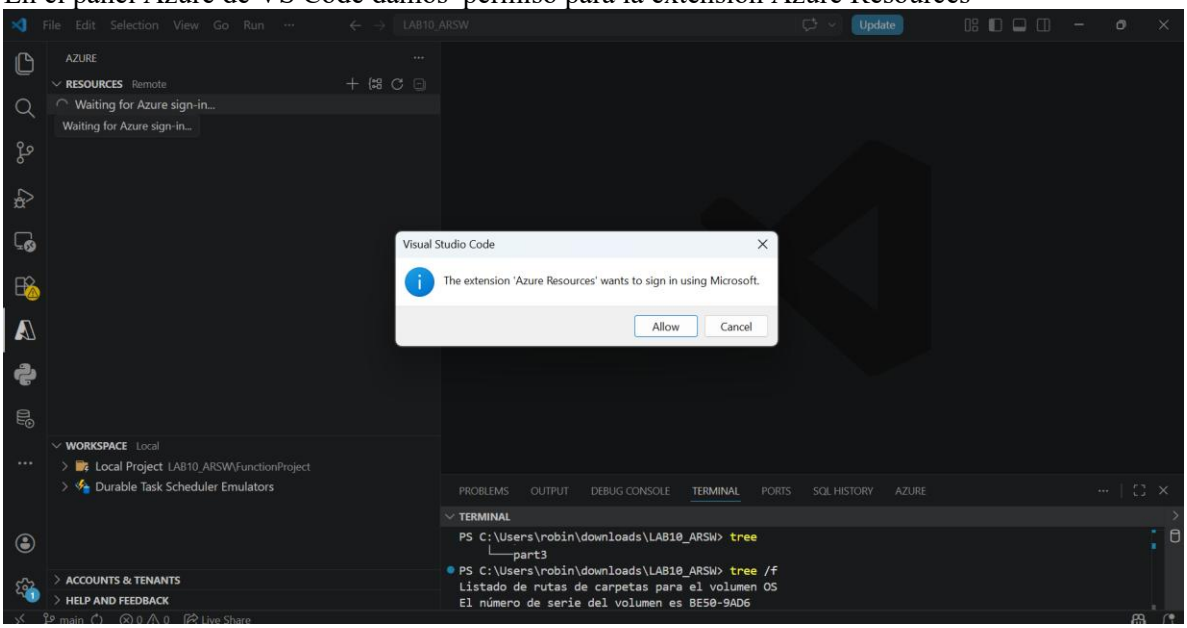


Parte 2: Extensión Azure Functions en Visual Studio Code

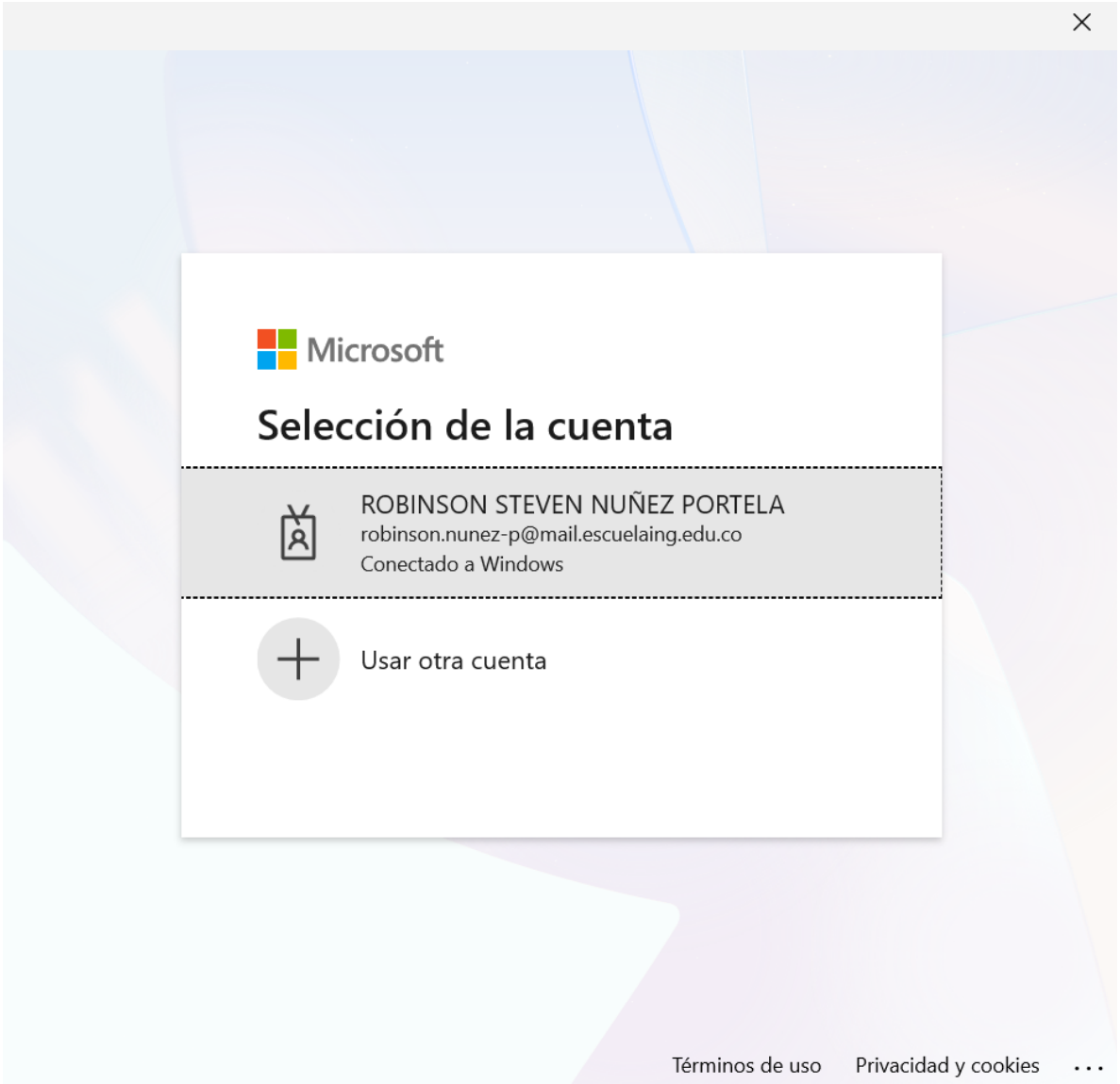
Para el desarrollo y despliegue local se utilizó Visual Studio Code con la extensión oficial de Azure Functions. Esta extensión permite autenticarse con la cuenta de Azure, gestionar Function Apps y publicar código directamente desde el IDE.



En el panel Azure de VS Code damos permiso para la extensión Azure Resources



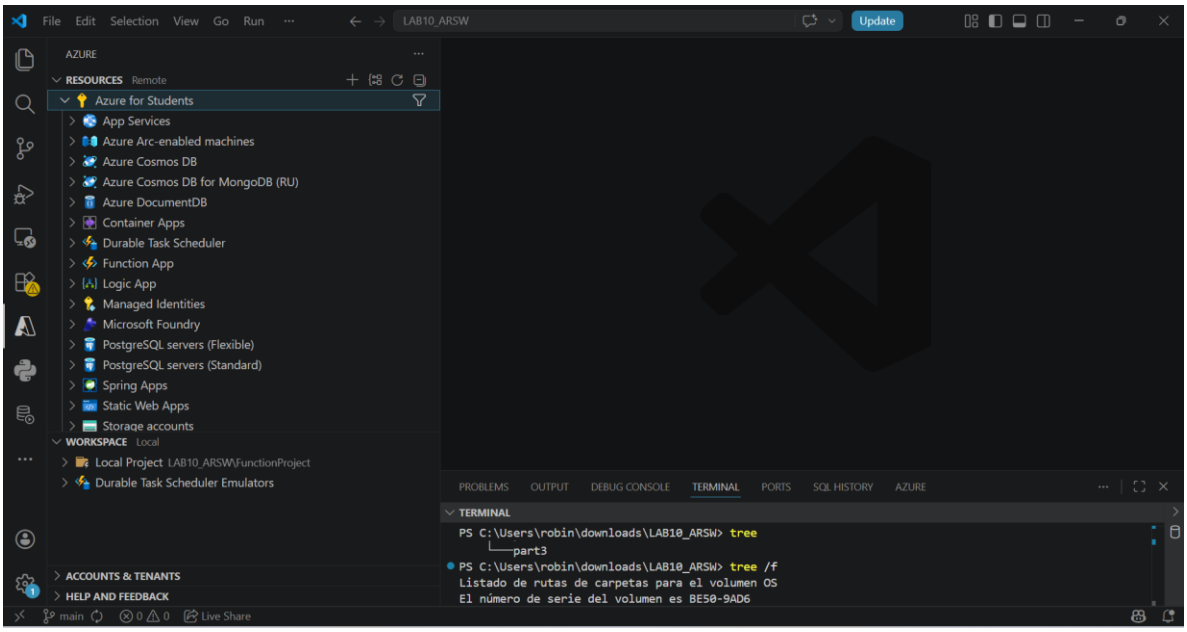
Se eligió la cuenta institucional robinson.nunez-p@mail.escuelaing.edu.co para autenticarse con Azure



Instalación exitosa de azure-functions-core-tools@4 mediante npm, y la verificación con 'func --version' que devuelve la versión 4.9.0.

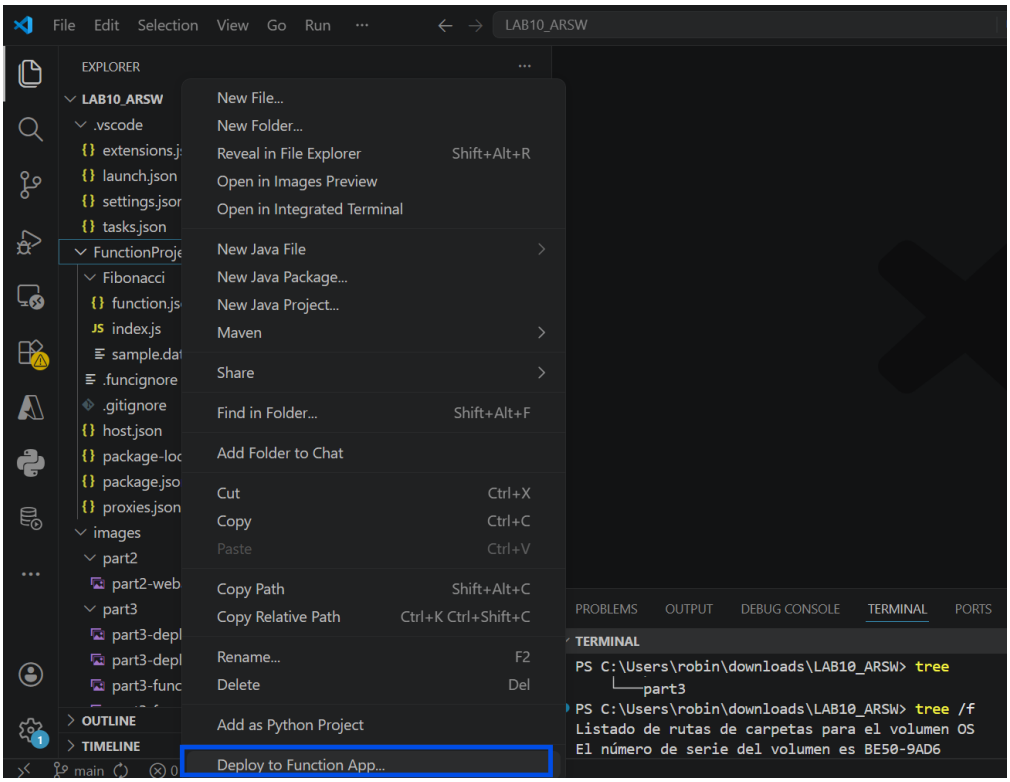
```
PS C:\WINDOWS\system32> npm install -g azure-functions-core-tools@4 --unsafe-perm true
npm warn Unknown cli config "--unsafe-perm". This will stop working in the next major version of npm.

changed 35 packages in 1m

5 packages are looking for funding
  run `npm fund` for details
PS C:\WINDOWS\system32> func --version
4.9.0
PS C:\WINDOWS\system32>
```

Panel AZURE de VS Code mostrando los recursos de la suscripción Azure for Students ya conectada.



Parte 3: Despliegue de la Función de Fibonacci

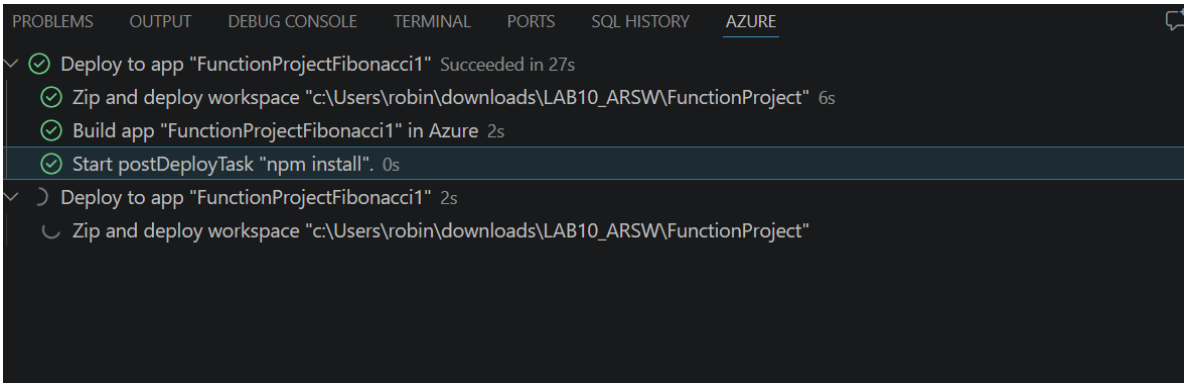
Se desplegó la función de cálculo del enésimo número de Fibonacci a la Function App creada en Azure. El despliegue se realizó directamente desde Visual Studio Code.



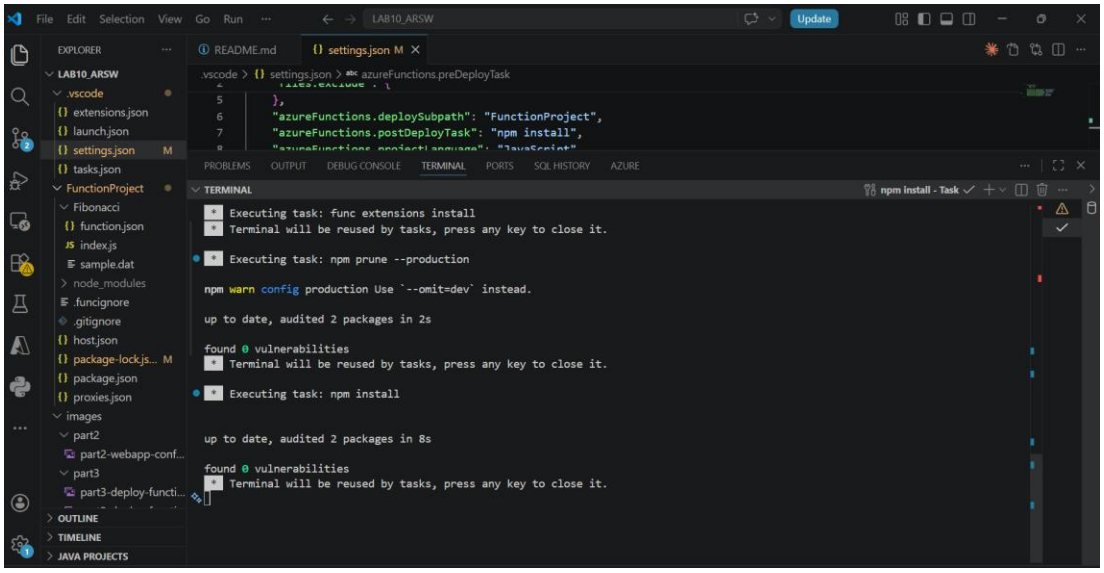
Se seleccionó FunctionProjectFibonacci1 del grupo de recursos SCALABILITY_LAB_II

Se modifico la línea azureFunctions.projectRuntim con 4 que define la versión del runtime ya que daba error con la versión 2.

Despliegue exitoso:



Todos los pasos finalizaron sin errores:



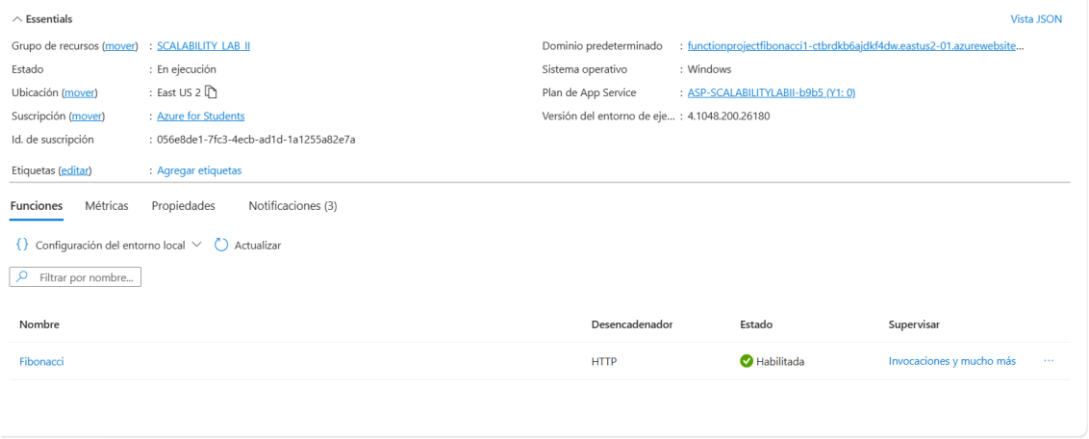
Parte 4: Prueba de la Función en el Portal de Azure

Una vez desplegada la función, se navegó al portal de Azure para realizar pruebas directas mediante la interfaz web. El portal ofrece una sección de prueba integrada donde se pueden enviar solicitudes HTTP y verificar la respuesta de la función.

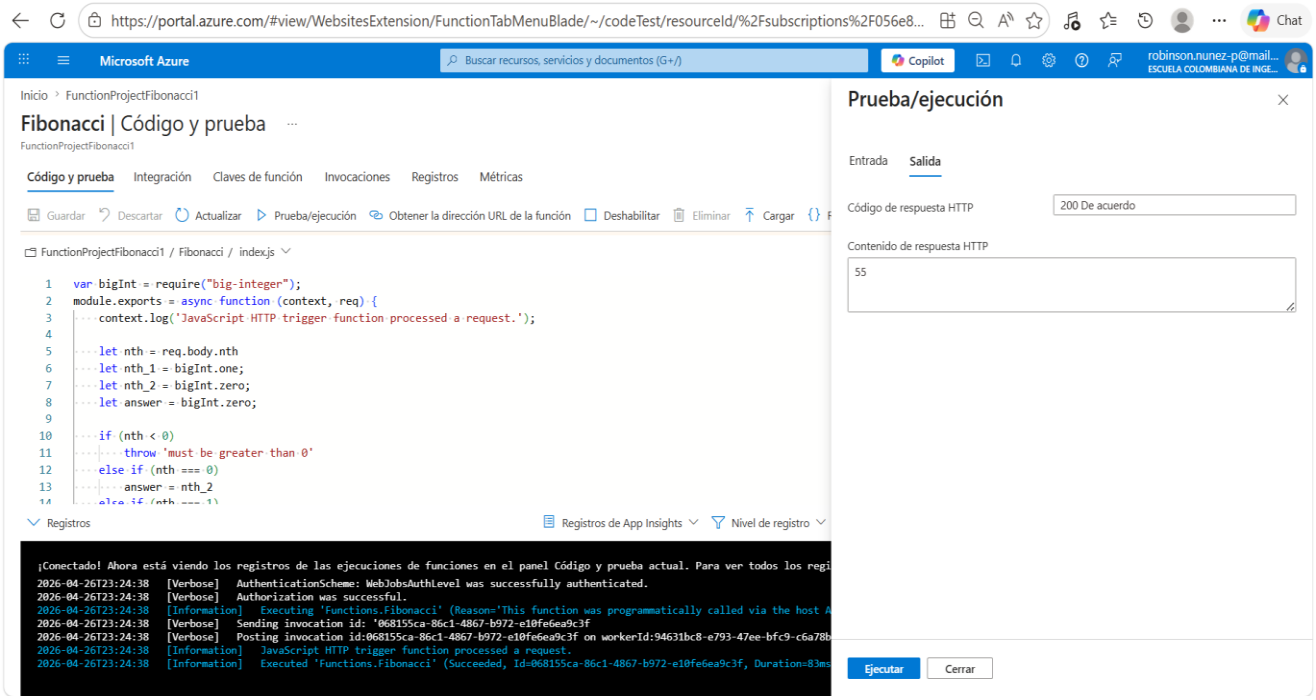
Se modifico la versión a [4.*, 5.0.0) ya que no funcionaba la dada por el laboratorio.

```
{
  "version": "2.0",
  "extensionBundle": {
    "id": "Microsoft.Azure.Functions.ExtensionBundle",
    "version": "[4.*, 5.0.0)"
  }
}
```

Portal Azure mostrando la función 'Fibonacci' con trigger HTTP en estado Habilitada



El portal de Azure muestra la respuesta de la función tras ejecutar la prueba. Se puede ver el código de estado HTTP 200, el tiempo de ejecución de la función y el valor calculado del enésimo número de Fibonacci retornado en el cuerpo de la respuesta, probando con el cuerpo nth: 10 que nos da 55.



Parte 5: Pruebas de Concurrencia con POSTMAN y NEWMAN

Se modificó la colección de POSTMAN con NEWMAN para enviar 10 peticiones concurrentes a la función de Fibonacci con valores superiores a 1.000.000. Esta prueba de carga permite verificar el comportamiento de la Function App bajo condiciones de uso simultáneo y evaluar el escalamiento automático del plan de consumo.

Primero instalamos 'npm install -g newman'.

```
PS C:\Users\robin\downloads\LAB10_ARSW> npm install -g newman
npm warn deprecated har-validator@5.1.5: this library is no longer supported
npm warn deprecated uuid@3.4.0: Please upgrade  to version 7 or higher.  Older versions may use Math.random() in
certain circumstances, which is known to be problematic.  See https://v8.dev/blog/math-random for details.
npm warn deprecated @faker-js/faker@5.5.3: Please update to a newer version.

added 148 packages in 49s

21 packages are looking for funding
  run `npm fund` for details
```

Se configuró la colección de POSTMAN para realizar múltiples peticiones en paralelo. Newman, la herramienta de línea de comandos para ejecutar colecciones de Postman, permite especificar el número de iteraciones paralelas mediante la opción --iteration-count.

```
{} fibonacci-collection.json > ...
1  {
2    "info": { "name": "Fibonacci", "schema": "https://schema.getpostman.com/json/collection/v2.1.0/collection.json" },
3    "item": [{
4      "name": "Fibonacci Request",
5      "request": {
6        "method": "POST",
7        "header": [{"key": "Content-Type", "value": "application/json"}],
8        "body": { "mode": "raw", "raw": "{\"nth\": 1000000}" },
9        "url": "https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci"
10     }
11   }]
12 }
```

Ejecución de Newman con el comando 'newman run fibonacci-collection.json -n 10'. Se observan las primeras 6 iteraciones respondiendo con código 200 OK en tiempos de ~12-13 segundo

```
PS C:\Users\robin\downloads\LAB10_ARSW> newman run fibonacci-collection.json -n 10
(node:21624) [DEP0176] DeprecationWarning: fs.F_OK is deprecated, use fs.constants.F_OK instead
(Use `node --trace-deprecation ...` to show where the warning was created)
newman

Fibonacci

Iteration 1/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 13s]

Iteration 2/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.4s]

Iteration 3/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.5s]

Iteration 4/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.7s]

Iteration 5/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.5s]

Iteration 6/10
```

Resultado final de las 10 iteraciones con Newman: 10 requests ejecutados, 0 fallidos, duración total 2m 7.6s y tiempo promedio de respuesta 12.6s.

```
PS C:\Users\robin\downloads\LAB10_ARSW> newman run fibonacci-collection.json -n 10
→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.5s]

Iteration 7/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.7s]

Iteration 8/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.6s]

Iteration 9/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.7s]

Iteration 10/10

→ Fibonacci Request
  POST https://functionprojectfibonacci1-ctbrdkb6ajdkf4dw.eastus2-01.azurewebsites.net/api/fibonacci [200 OK, 209.23kB, 12.7s]
```

	executed	failed
iterations	10	0
requests	10	0
test-scripts	0	0
prerequest-scripts	0	0
assertions	0	0
total run duration: 2m 7.6s		
total data received: 2.09MB (approx)		
average response time: 12.6s [min: 12.4s, max: 13s, s.d.: 147ms]		

```
PS C:\Users\robin\downloads\LAB10_ARSW>
```

Los resultados muestran cómo Azure Functions escala automáticamente para atender las solicitudes concurrentes. El sistema distribuyó la carga entre múltiples instancias, garantizando que todas las peticiones fueran respondidas satisfactoriamente.

Parte 6: Función Fibonacci con Memoization

Se implementó una nueva versión de la función Fibonacci utilizando un enfoque recursivo con memoization. Esta técnica almacena los resultados de subproblemas ya calculados en un mapa en memoria para evitar cálculos redundantes y mejorar el rendimiento en llamadas sucesivas.

Archivo function.json de la función FibonacciMemo:

FunctionProject > FibonacciMemo > {} function.json > ...

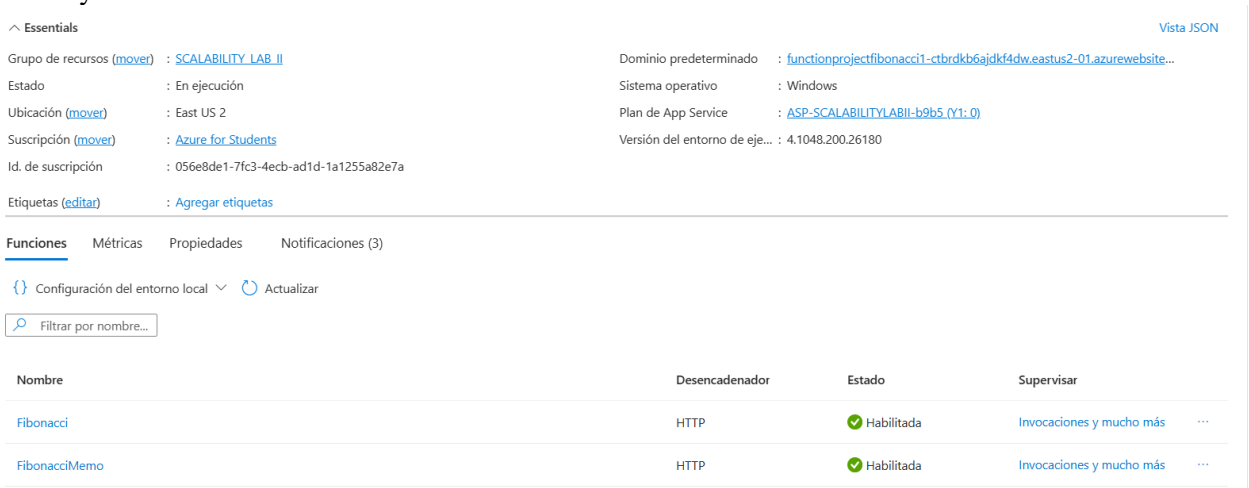
```
1   {
2     "bindings": [
3       {
4         "authLevel": "anonymous",
5         "type": "httpTrigger",
6         "direction": "in",
7         "name": "req",
8         "methods": ["get", "post"]
9       },
10      {
11        "type": "http",
12        "direction": "out",
13        "name": "res"
14      }
15    ]
16  }
```

Código fuente index.js de FibonacciMemo almacena resultados para evitar recálculos.

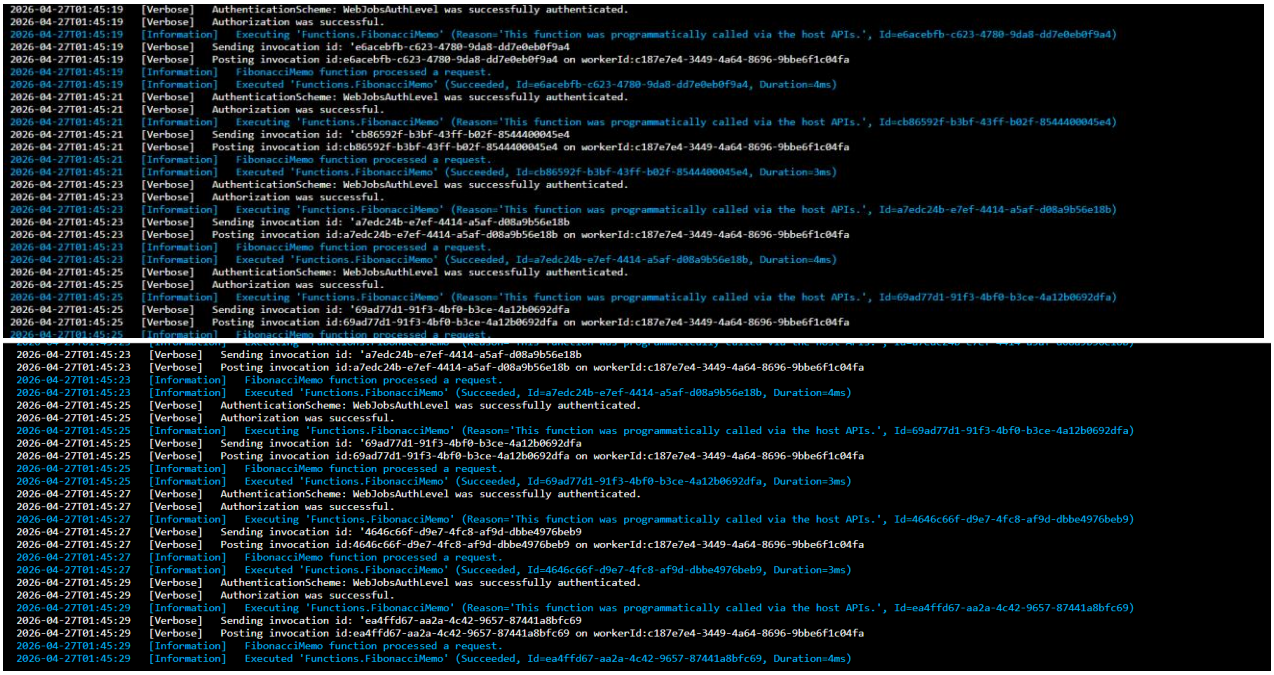
FunctionProject > FibonacciMemo > JS index.js > <unknown> > exports

```
1   var bigInt = require("big-integer");
2
3   let memo = {};
4
5   function fibMemo(n) {
6       if (n in memo) return memo[n];
7       if (n <= 0) return bigInt.zero;
8       if (n === 1) return bigInt.one;
9       memo[n] = fibMemo(n - 1).add(fibMemo(n - 2));
10      return memo[n];
11  }
12
13  module.exports = async function (context, req) {
14      context.log('FibonacciMemo function processed a request.');
15      let nth = req.body.nth;
16      if (nth < 0) throw 'must be greater than 0';
17      context.res = { body: fibMemo(nth).toString() };
18  }
```

Portal Azure con las dos funciones desplegadas: Fibonacci' y FibonacciMemo, ambas con trigger HTTP y estado Habilitada.



Varias pruebas con {"nth": 50}



Después de 5 minutos se evidencio que no pasaba nada así que se espero unos 20 minutos:

```
2026-04-27T01:45:29  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=ea4ffd67-aa2a-4c42-9657-87441a8bfc69, Duration=4ms)
2026-04-27T01:50:34  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:34  [Verbose]   Authorization was successful.
2026-04-27T01:50:34  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=251a9311-6840-4162-b92f-25a11a48fb64)
2026-04-27T01:50:34  [Verbose]   Sending invocation id: '251a9311-6840-4162-b92f-25a11a48fb64
2026-04-27T01:50:34  [Verbose]   Posting invocation id:251a9311-6840-4162-b92f-25a11a48fb64 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:34  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:34  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=251a9311-6840-4162-b92f-25a11a48fb64, Duration=4ms)
2026-04-27T01:50:36  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:36  [Verbose]   Authorization was successful.
2026-04-27T01:50:36  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=3d75c56c-222b-4f04-812b-758503e011b2)
2026-04-27T01:50:36  [Verbose]   Sending invocation id: '3d75c56c-222b-4f04-812b-758503e011b2
2026-04-27T01:50:36  [Verbose]   Posting invocation id:3d75c56c-222b-4f04-812b-758503e011b2 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:36  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:36  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=3d75c56c-222b-4f04-812b-758503e011b2, Duration=3ms)
2026-04-27T01:50:38  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:38  [Verbose]   Authorization was successful.
2026-04-27T01:50:38  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=7e5f5928-cac0-41de-9ca3-27018902af11)
2026-04-27T01:50:38  [Verbose]   Sending invocation id: '7e5f5928-cac0-41de-9ca3-27018902af11
2026-04-27T01:50:38  [Verbose]   Posting invocation id:7e5f5928-cac0-41de-9ca3-27018902af11 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:38  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:38  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=7e5f5928-cac0-41de-9ca3-27018902af11, Duration=2ms)
2026-04-27T01:50:40  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:40  [Verbose]   Authorization was successful.
2026-04-27T01:50:40  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=375c905b-38a4-4c70-aba8-bd6443a1375b)
2026-04-27T01:50:40  [Verbose]   Sending invocation id: '375c905b-38a4-4c70-aba8-bd6443a1375b
```

```
2026-04-27T01:50:34  [Verbose]   Sending invocation id: '251a9311-6840-4162-b92f-25a11a48fb64
2026-04-27T01:50:34  [Verbose]   Posting invocation id:251a9311-6840-4162-b92f-25a11a48fb64 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:34  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:34  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=251a9311-6840-4162-b92f-25a11a48fb64, Duration=4ms)
2026-04-27T01:50:36  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:36  [Verbose]   Authorization was successful.
2026-04-27T01:50:36  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=3d75c56c-222b-4f04-812b-758503e011b2)
2026-04-27T01:50:36  [Verbose]   Sending invocation id: '3d75c56c-222b-4f04-812b-758503e011b2
2026-04-27T01:50:36  [Verbose]   Posting invocation id:3d75c56c-222b-4f04-812b-758503e011b2 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:36  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:36  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=3d75c56c-222b-4f04-812b-758503e011b2, Duration=3ms)
2026-04-27T01:50:38  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:38  [Verbose]   Authorization was successful.
2026-04-27T01:50:38  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=7e5f5928-cac0-41de-9ca3-27018902af11)
2026-04-27T01:50:38  [Verbose]   Sending invocation id: '7e5f5928-cac0-41de-9ca3-27018902af11
2026-04-27T01:50:38  [Verbose]   Posting invocation id:7e5f5928-cac0-41de-9ca3-27018902af11 on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:38  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:38  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=7e5f5928-cac0-41de-9ca3-27018902af11, Duration=2ms)
2026-04-27T01:50:40  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T01:50:40  [Verbose]   Authorization was successful.
2026-04-27T01:50:40  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=375c905b-38a4-4c70-aba8-bd6443a1375b)
2026-04-27T01:50:40  [Verbose]   Sending invocation id: '375c905b-38a4-4c70-aba8-bd6443a1375b
2026-04-27T01:50:40  [Verbose]   Posting invocation id:375c905b-38a4-4c70-aba8-bd6443a1375b on workerId:c187e7e4-3449-4a64-8696-9bbe6f1c04fa
2026-04-27T01:50:40  [Information]   FibonacciMemo function processed a request.
2026-04-27T01:50:40  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=375c905b-38a4-4c70-aba8-bd6443a1375b, Duration=3ms)
```

Tras esperar unos 20 minutos se evidencio esto:

```
2026-04-27T02:10:01  [Information]   FibonacciMemo function processed a request.
2026-04-27T02:10:01  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=6ccc8648-d850-4710-bb0c-79e4b58cd372, Duration=4ms)
2026-04-27T02:14:06  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T02:14:06  [Verbose]   Authorization was successful.
2026-04-27T02:14:06  [Information]   Executing 'Functions.FibonacciMemo' (Reason='This function was programmatically called via the host APIs.', Id=79de8eb2-b296-4d38-9169-c4b91de4382e)
2026-04-27T02:14:06  [Verbose]   Sending invocation id: '79de8eb2-b296-4d38-9169-c4b91de4382e
2026-04-27T02:14:06  [Verbose]   Posting invocation id:79de8eb2-b296-4d38-9169-c4b91de4382e on workerId:8679dd09-a8f6-4213-8846-fe7fb62d2d49
2026-04-27T02:14:06  [Information]   FibonacciMemo function processed a request.
2026-04-27T02:14:06  [Information]   Executed 'Functions.FibonacciMemo' (Succeeded, Id=79de8eb2-b296-4d38-9169-c4b91de4382e, Duration=21ms)
2026-04-27T02:14:08  [Verbose]   AuthenticationScheme: WebJobsAuthLevel was successfully authenticated.
2026-04-27T02:14:08  [Verbose]   Authorization was successful.
```

¿Cuál es el comportamiento?

Comparación final de los tiempos de respuesta registrados antes y después de los 5 minutos de inactividad. Los datos confirman que la memoization en memoria no es una estrategia persistente en entornos serverless con el Plan de Consumo de Azure Functions.

Este comportamiento confirma que Azure Functions en el Plan de Consumo destruye las instancias inactivas cold start, lo que elimina cualquier estado en memoria, incluyendo el mapa de memoization.

Preguntas y Respuestas

¿Qué es una Azure Function?

Una Azure Function es un servicio de cómputo serverless de Microsoft Azure que permite ejecutar pequeños fragmentos de código denominados funciones en respuesta a eventos, sin necesidad de aprovisionar ni gestionar infraestructura de forma explícita. Estos eventos pueden ser de distintos tipos: solicitudes HTTP, mensajes en una cola como Service Bus, Storage Queue, temporizadores programados cron, cambios en bases de datos de Cosmos DB, eventos de Blob Storage, entre otros.

Cada función se define en un lenguaje de programación compatible entre estos C#, JavaScript/TypeScript, Python, Java, PowerShell, Go junto con un conjunto de triggers y bindings enlazados de entrada y salida. Azure se encarga de gestionar automáticamente el ciclo de vida, la disponibilidad y el escalamiento de las instancias que ejecutan ese código.

¿Qué es Serverless?

Serverless es un modelo de ejecución y facturación en la nube en el que el proveedor cloud gestiona completamente la infraestructura subyacente servidores físicos, sistemas operativos, parches, escalamiento de manera transparente para el desarrollador. El término sin servidor es una abstracción: Los servidores existen, pero el desarrollador no necesita preocuparse por ellos.

Las características principales del modelo serverless son:

- Escalamiento automático: El proveedor lanza y elimina instancias según la demanda real, en cuestión de milisegundos.
- Facturación por ejecución: Se paga únicamente por el tiempo de cómputo y los recursos consumidos durante la ejecución de las funciones, no por instancias activas permanentemente.
- Sin gestión de infraestructura: El equipo de desarrollo solo escribe código de negocio; la operación queda en manos del proveedor.
- Stateless por diseño: Cada invocación es independiente. El estado persistente debe gestionarse en servicios externos ya sean bases de datos, caches o colas.

¿Qué es el Runtime y qué implica seleccionarlo?

El runtime es el entorno de ejecución que interpreta o compila y ejecuta el código de la función. En Azure Functions, el runtime define: El lenguaje de programación disponible en Node.js, Python, .NET, Java, etc… La versión de ese lenguaje, las bibliotecas del SDK de Azure Functions disponibles, y la compatibilidad con determinadas versiones del host de Functions.

Seleccionarlo al momento de crear la Function App implica:

- El lenguaje queda fijo: No se puede cambiar sin recrear la Function App.
- Las dependencias y paquetes disponibles están determinados por la versión del runtime.
- La compatibilidad con triggers y bindings puede variar entre versiones del runtime.
- Actualizaciones del runtime pueden requerir ajustes en el código si introducen cambios incompatibles.

Por ello, se recomienda escoger la versión de runtime más reciente y estable al momento del desarrollo para aprovechar las últimas características y el soporte más largo.

¿Por qué es necesario crear un Storage Account junto con una Function App?

Azure Functions utiliza internamente el Storage Account para múltiples propósitos críticos de su funcionamiento:

- Estado del runtime: El host de Functions almacena metadatos sobre el estado de ejecución, versiones y configuración en Blob Storage y Table Storage.
- Triggers y bindings: Algunos triggers como el Timer Trigger almacenan su estado de bloqueo y punto de control en el Storage Account para garantizar ejecución única y ordenada.
- Logs y diagnóstico: Los registros de ejecución y errores se escriben en el Storage Account antes de enviarse a Application Insights u otros destinos.
- Coordinación de escalamiento: En el Plan de Consumo, el controlador de escala usa el Storage Account para coordinar qué instancias están activas y distribuir la carga.

Sin un Storage Account asociado, el runtime de Azure Functions no puede inicializarse correctamente. Por esta razón, Azure exige su creación o vinculación a uno existente como paso obligatorio al crear una Function App.

¿Cuáles son los tipos de planes para una Function App? Diferencias, ventajas y desventajas.

Azure Functions ofrece tres planes de hospedaje principales, cada uno con distintas características de escalamiento, coste y rendimiento:

Plan	Ventajas	Desventajas
Consumo (Serverless)	Pago estrictamente por uso. Escala automática de 0 a N instancias. Primer millón de ejecuciones gratuito por mes. Cero configuraciones de infraestructura.	Cold start: latencia adicional al despertar instancias inactivas. Límite de 10 min por ejecución. No soporta VNet privada en la versión básica.
Premium (Elastic Premium)	Sin cold start (instancias pre-calentadas). Soporte de VNet Integration. Mayor potencia de CPU y memoria. Escala automática con mínimo garantizado.	Coste fijo mensual por la instancia mínima siempre activa. Más costoso que el plan de consumo para cargas bajas o esporádicas.
App Service (Dedicated)	Control total de recursos. Sin cold start. Sin límite de tiempo de ejecución. Se puede correr en máquinas dedicadas. Permite Always On.	Se paga por la capacidad reservada, independientemente del uso real. No escala automáticamente (requiere configuración manual de autoscale). Mayor complejidad operativa.

¿Por qué la memoization falla o no funciona correctamente?

La memoization almacena los resultados de cálculos previos en una estructura de datos en memoria, por ejemplo, un Map o Dictionary para evitar recalcularlos en invocaciones posteriores. Sin embargo, en el contexto de Azure Functions en el Plan de Consumo, este mecanismo falla por las siguientes razones:

- Stateless por diseño: Azure Functions no garantiza la persistencia del estado entre invocaciones. Cada ejecución puede correr en una instancia nueva o en una instancia existente, pero no hay garantías.
- Cold start y destrucción de instancias: Cuando la función no recibe tráfico por un período de tiempo aproximadamente 5 minutos en el Plan de Consumo, Azure destruye las instancias inactivas para liberar recursos. Al destruirse la instancia, toda la memoria que contenía el mapa de memoization se pierde irreversiblemente.
- Múltiples instancias en paralelo: Cuando hay concurrencia alta, Azure puede crear múltiples instancias en paralelo. El caché de una instancia no es visible para las demás, por lo que no hay un caché compartido global.

Solución: para implementar un caché persistente y compartido en Azure Functions se debe usar un servicio externo como Azure Cache for Redis, Azure Cosmos DB o Azure Table Storage, que sobreviva al ciclo de vida de las instancias.

¿Cómo funciona el sistema de facturación de las Function App?

El modelo de facturación varía según el plan elegido:

Plan de Consumo

La facturación se basa en dos métricas principales:

- Número de ejecuciones: el primer millón de ejecuciones por mes es gratuito. A partir de ese umbral, se cobra aproximadamente $0.20 USD por cada millón de ejecuciones adicionales.
- Consumo de recursos GB-segundos: se mide como el producto de la memoria utilizada en GB por el tiempo de ejecución en segundos. El primer 400.000 GB-s por mes es gratuito. El excedente se cobra a aproximadamente $0.000016 USD por GB-s.

Plan Premium Elastic Premium:

Se factura por hora según el número de vCore-segundos y GB-segundos consumidos por las instancias activas (incluyendo la instancia mínima pre-calentada). No hay capa gratuita; se paga desde el primer segundo.

Plan App Service Dedicated:

Se factura según el plan de App Service subyacente B1, S1, P1v2, etc. De manera fija mensual, independientemente del uso real de las funciones. Es el modelo más predecible en coste, pero el menos eficiente para cargas variables o bajas.

Informe de Resultados

Resumen de la experiencia de laboratorio

A lo largo de este laboratorio se implementó y desplegó exitosamente una función serverless en Azure Functions para calcular el enésimo número de la secuencia de Fibonacci. El desarrollo abarcó desde la creación de los recursos en el portal de Azure hasta las pruebas de concurrencia con Newman y la comparación de enfoques algorítmicos.

Pruebas de Concurrencia, Análisis:

Al ejecutar 10 peticiones concurrentes con Newman sobre la función iterativa de Fibonacci, se observó lo siguiente:

- Azure Functions escaló automáticamente, distribuyendo las solicitudes entre múltiples instancias para mantener tiempos de respuesta aceptables.
- El consumo de CPU se mantuvo por debajo del umbral crítico del 70% gracias al escalamiento horizontal automático del plan de consumo.
- Todas las peticiones fueron atendidas y respondidas correctamente, cumpliendo con el requisito de escalabilidad del escenario de calidad planteado.
- Los tiempos de respuesta para valores altos de n superiores a 1.000.000)fueron considerablemente más altos que para valores pequeños, lo cual es esperable dada la complejidad computacional O(n) del algoritmo iterativo.

Comportamiento de la Memoization

La función con memoization demostró un rendimiento significativamente superior en las primeras invocaciones del ciclo de vida de la instancia: Los cálculos subsecuentes de valores ya computados retornaban respuesta casi instantánea al recuperar el resultado del mapa en memoria.

Sin embargo, tras un período de inactividad de 5 minutos, la instancia fue destruida por Azure comportamiento esperado del Plan de Consumo. Al retomar las peticiones, el cold start reinicializó la instancia desde cero, perdiéndose el mapa de memoization por completo. Las respuestas volvieron a los tiempos de la primera ejecución, evidenciando la naturaleza stateless del modelo serverless.

Conclusiones

- Azure Functions es una herramienta poderosa para implementar lógica de negocio escalable y de bajo coste en escenarios de carga variable.
- El Plan de Consumo es ideal para prototipos, cargas esporádicas y entornos con requerimientos de costo mínimo, pero presenta limitaciones de cold start y ausencia de estado persistente.
- La memoization en memoria no es una estrategia viable para entornos serverless sin un backend de caché externo como Redis y CosmosDB. El estado debe externalizarse siempre.
- El escalamiento automático de Azure Functions garantizó la disponibilidad de todas las peticiones concurrentes, demostrando el cumplimiento del requisito de calidad de escalabilidad establecido.
- Para aplicaciones con latencia crítica o que requieren estado persistente, el Plan Premium o un App Service Plan dedicado son opciones más adecuadas, a pesar de su mayor costo.